Page :
Date :

EXPERIMENT - 07

d | Cheapest flight from source to destination with atmost k-stops.

Algo:

① Build the graph → convert flight list into adjacency list.

② Initialize BFS queue → Each element: {current city, total cost so far, stops used}
↳ starts with {source, 0, 0}

③ Track minimum cost: create minimumCost[n] which will store cheapest/minimum cost to reach each city.

④ BFS Traversal:

while queue is not empty, we pop the element from front {city, cost, stops}. If stops > k, skip it and for each neighbour,
↳ if city + price < minCost[next city], then update $ and push into the queue

⑤ Return the result

- If minCost[dst] is still INT_MAX → return -1 (not reachable)
- Else → return minCost[dst] (cheapest cost)

Key Idea: Here we use BFS.

Code on next page.

Name: Gurshaan

Page :
Date :

Experiment 07

Time Complexity: O(E * k)
Space Complexity: O(E+V)

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;

class Solution {
  public:
    int findCheapestPrice(int n, vector<vector<int>>& flights, int src,
int dst, int k) {
        for(auto & flight: flights){
            graph[flight[0]].push_back({flight[1], flight[2]});
        }

        queue<vector<int>> q;
        q.push({src, 0, 0});
        vector<int> minCost(n, INT_MAX);
        minCost[src] = 0;
        while(!q.empty()) {
            auto current = q.front();
            q.pop();
            int city = current[0], cost = current[1], stops =
current[2];
            if(auto & neighbour: graph[city]){
                int nextCity = neighbour.first;
                int price = neighbour.second;
                if(cost + price < minCost[nextCity]) {
                    minCost[nextCity] = cost + price;
                    q.push({nextCity, cost+price, stops+1});
                }
            }
        }
    return minCost[d] == INT_MAX ? -1: minCost[d];
    }
```